
JavaScript: do básico ao DOM

Aula de reforço e revisão antes do projeto Lista de Tarefas

```
console.log("Bem-vindo à revisão!");
```

| Mapa da revisão técnica profunda

TÓPICOS DE ESTUDO

JS no browser: script e console

Tipos de dados, const/let

Template string e Métodos de string

Números, Math e conversões

Arrays: iteração, busca, map e filter

DOM: seleção, atributos e classes

Eventos: addEventListener e event object

Formulários: captura e validação

OBJETIVO DA REVISÃO

Análise Conceitual

Não apenas "como fazer", mas "por que funciona". Entendendo o comportamento interno da linguagem.

Detalhamento Técnico

Explicações profundas sobre escopo, tipos de referência, árvore do DOM e ciclo de vida de eventos.

Base para Projetos

Consolidar o conhecimento necessário para construir qualquer interface interativa moderna.

1. O Ecossistema do Browser

O Papel da Engine

O JavaScript não roda "sozinho". Ele depende de uma **Engine** (como a V8 do Chrome) que traduz o código em linguagem de máquina. O browser fornece o **Runtime**, que inclui a Web API (DOM, Fetch, Timer).

Fluxo de Execução

O JS é **Single-Threaded** e síncrono por padrão. Ele lê o script de cima para baixo, linha por linha. Se um script pesado trava, a interface do usuário também trava.

O Console como REPL

O console do desenvolvedor é um ambiente **Read-Eval-Print Loop**. Ele permite inspecionar o estado atual da memória, testar seletores do DOM e depurar o fluxo de dados em tempo real.

```
// Inspeccionando o objeto global
console.log(window);

// Medindo performance
console.time('loop');
// ... código ...
console.timeEnd('loop');
```

2. Variáveis e Tipagem Dinâmica

Escopo e Temporal Dead Zone

Diferente de

```
var
```

, as palavras-chave

```
let
```

e

```
const
```

possuem **escopo de bloco**. Isso evita vazamentos de variáveis e torna o código mais previsível.

```
const
```

não significa que o valor é imutável, mas sim que a **referência** não pode ser alterada. Objetos e Arrays declarados com

```
const
```

podem ter seus conteúdos modificados.

```
const user = { name: "Ana" };  
  
user.name = "Bia"; // Permitido  
  
user = {}; // Erro de atribuição
```

Tipos Primitivos vs Referência

JavaScript possui 7 tipos primitivos (String, Number, Boolean, Null, Undefined, Symbol, BigInt). Eles são passados por **valor**.

Objetos, Arrays e Funções são tipos de **referência**. Ao copiar um objeto, você copia o endereço de memória, não os dados em si.

```
let a = 10; let b = a; b = 20; // a continua 10
```

```
let arr1 = [1]; let arr2 = arr1;  
  
arr2.push(2); // arr1 agora é [1, 2]
```

3. Manipulação de Dados Primitivos

Strings: Mais que Texto

Strings em JS são imutáveis. Métodos como `replace` ou `toUpperCase` retornam uma **nova string**.

Template Strings: Permitem interpolação de expressões e strings multi-linha, facilitando a criação de fragmentos de HTML via JS.

```
const nome = "Manus";
const html = `
  <div>
    Olá, ${nome.trim()}
  </div>
`;
```

Números e Coerção

JavaScript é fracamente tipado. A **coerção de tipo** ocorre quando o JS tenta converter valores automaticamente (ex: `"5" - 2 = 3`).

O objeto `Math` fornece métodos estáticos para operações complexas, essencial para cálculos de interface e animações.

```
const preco = "49.90";
const num = parseFloat(preco);

// Arredondamento inteligente
Math.ceil(num); // 50
Math.floor(num); // 49
Math.random(); // 0 a 0.999 ...
```

4. Arrays e Métodos de Alta Ordem

A Anatomia do Array

Arrays em JS são objetos especializados. Eles possuem a propriedade

```
length
```

e métodos que permitem manipular dados de forma eficiente.

Mutação vs Imutabilidade: Métodos como

```
push
```

e

```
splice
```

alteram o array original. Métodos modernos como

```
map
```

e

```
filter
```

criam novos arrays.

```
const nums = [1, 2, 3];  
// Mutação  
nums.push(4);  
// Imutabilidade (Spread Operator)  
const novos = [...nums, 5];
```

Programação Declarativa

Em vez de usar loops

```
for
```

manuais, usamos métodos declarativos que descrevem **o que** queremos, não **como** fazer.

```
map
```

: Transforma cada elemento.

```
filter
```

: Remove elementos indesejados.

```
find
```

: Localiza o primeiro item que satisfaz uma condição.

```
const tarefas = [  
  { id: 1, status: 'ok' },  
  { id: 2, status: 'pendente' }  
];  
const pendentes = tarefas.filter(t => t.status === 'pendente');
```

5. Funções e Cidadãos de Primeira Classe

First-Class Citizens

Em JavaScript, funções são objetos. Elas podem ser armazenadas em variáveis, passadas como argumentos e retornadas por outras funções.

Arrow Functions: Sintaxe curta que não possui seu próprio `this`, sendo ideais para métodos de array e callbacks de eventos.

```
// Declaração clássica
function soma(a, b) { return a + b; }

// Arrow function
const soma = (a, b) => a + b;
```

O Poder do Callback

Um **Callback** é uma função passada para outra para ser executada "depois". Isso é a base da programação assíncrona e da interatividade no browser.

Sem callbacks, não poderíamos reagir a cliques ou esperar por dados de uma API sem travar o browser.

```
const log = () => console.log("Fui!");

// Passando a função como referência
setTimeout(log, 2000);
```

6. DOM: A Ponte entre JS e HTML

O que é o DOM?

O **Document Object Model** é uma representação em árvore do seu HTML. Cada tag vira um **Objeto** que o JavaScript pode ler e alterar.

Ao mudar uma propriedade no JS, o browser detecta a mudança e faz o **Reflow/Repaint** da tela automaticamente.

```
const titulo = document.querySelector('h1');
titulo.style.color = 'red';
titulo.innerText = 'Novo Título';
```

Manipulação Estrutural

Podemos criar elementos do zero com

```
createElement
```

e injetá-los na árvore com

```
appendChild
```

ou

```
prepend
```

classList: A melhor forma de alterar a aparência é alternando classes CSS, mantendo a separação entre lógica (JS) e estilo (CSS).

```
const li = document.createElement('li');
li.classList.add('item-lista', 'ativo');
lista.appendChild(li);
```

7. Eventos e Propagação

Escutando o Usuário

O `addEventListener` registra uma função para ser chamada quando um evento específico ocorre. É possível registrar múltiplos ouvintes para o mesmo evento.

Event Object: O primeiro argumento do callback contém dados cruciais: qual tecla foi apertada, a posição do mouse ou o elemento exato que disparou o evento.

```
input.addEventListener('keydown', e => {  
  if (e.key === 'Enter') {  
    console.log("Usuário deu Enter!");  
  }  
});
```

Bubbling (Propagação)

Eventos em JS "borbulham". Se você clica em um botão dentro de uma `div`, o evento de clique dispara no botão e depois na `div`.

Isso permite a **Delegação de Eventos**: colocar um único ouvinte no pai para controlar todos os filhos.

```
lista.addEventListener('click', e => {  
  if (e.target.tagName === 'LI') {  
    e.target.remove();  
  }  
});
```

8. Formulários e Fluxo de Dados

Interceptando o Envio

Por padrão, formulários HTML recarregam a página ao serem enviados. Em aplicações modernas, usamos

```
event.preventDefault()
```

para impedir isso e processar os dados via JS.

Sanitização: Sempre use

```
trim()
```

para remover espaços em branco extras e valide se o campo não está vazio antes de processar.

```
form.addEventListener('submit', e => {  
  e.preventDefault();  
  const data = input.value.trim();  
  if (data) process(data);  
});
```

Feedback e UX

A validação não deve apenas impedir o erro, mas informar o usuário. Manipulamos o DOM para mostrar mensagens de erro ou sucesso em tempo real.

Reset: Após um envio bem-sucedido, use

```
form.reset()
```

para limpar todos os campos e preparar a interface para a próxima entrada.

```
if (!valido) {  
  msg.innerText = "Campo obrigatório";  
  msg.classList.add('erro');  
}
```

9. Arquitetura e Separação de Preocupações

O Conceito de Estado

O **Estado** é a "verdade" da sua aplicação em um dado momento. Em vez de ler dados diretamente do HTML, devemos manter os dados em variáveis JS (como um Array de Objetos).

A interface deve ser apenas um reflexo desse estado. Se o estado muda, a interface é atualizada.

Responsabilidades Claras

Lógica de Negócio: Funções que calculam, filtram ou alteram dados. Não devem saber que o HTML existe.

Lógica de Interface: Funções que escutam eventos e manipulam o DOM. Elas chamam a lógica de negócio e atualizam a tela.

```
// Lógica de Negócio
const add = (list, item) => [...list, item];

// Lógica de Interface
const handleAdd = () => {
  state = add(state, input.value);
  render();
};
```

Conclusão e Mentalidade de Engenharia

Entenda a Base

A sintaxe é apenas a ponta do iceberg. Entender escopo, tipos de referência e o ciclo de eventos é o que diferencia um codificador de um engenheiro.

DOM é uma Interface

Trate o DOM como uma camada de visualização. Mantenha sua lógica de dados separada para que seu código seja fácil de testar e manter.

Depuração Constante

Use o console e as ferramentas de desenvolvedor. Erros não são falhas, são sinais que indicam onde seu modelo mental da linguagem precisa de ajuste.

Próximos Passos

Com esta base sólida, você está pronto para explorar APIs assíncronas, frameworks modernos e aplicações complexas de página única (SPA).

> Revisão concluída. O conhecimento agora é seu.